

Test 01 Written Questions

530 Comprehensive Exam Practice

Derek Harter

Department of Computer Science
East Texas A&M University

Fall 2026



EAST TEXAS A&M

Virtual Address Translation (RISC-V Sv39)

530 Comprehensive Exam Practice

Derek Harter

Department of Computer Science
East Texas A&M University

Fall 2026



EAST TEXAS A&M

RISC-V Paging Hardware (review)

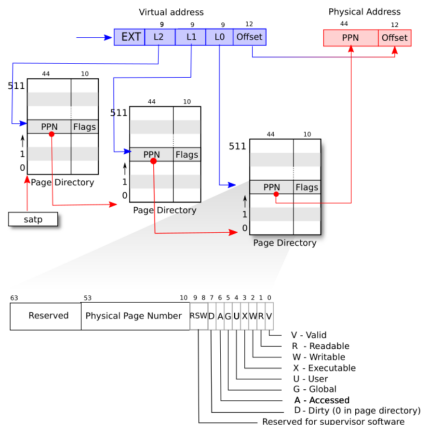


Figure 1: RISC-V address translation details (Cox et al., 2026, pg.33)

- RISC-V instructions (both user and kernel) manipulate virtual addresses
- The machine's RAM, or physical memory, is indexed with physical addresses
- The RISC-V page table hardware maps each virtual address to physical address
- Xv6 uses RISC-V Sv39 mode
 - only bottom 39 bits (of 64) are used
 - RISC-V page table is array of 2^{27} (134,217,728) **page table entries** PTEs
 - Each PTE contains 44-bit physical page number PPN and flags
 - Paging hardware translates top 27 bits of virtual address into page table index to find PTE
 - Results in 56-bit physical address, 44-bits from PPN, and 12-bit offset



RISC-V Paging Hardware (review 2)

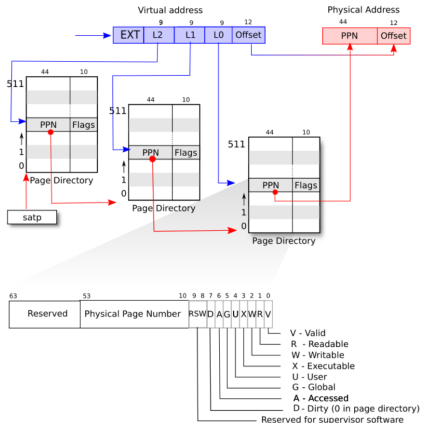


Figure 2: RISC-V address translation details (Cox et al., 2026, pg.33)

- A page table gives the operating system control over virtual-to-physical address translations
 - page size $2^{12} = 4096$ bytes `kernel/riscv.h PGSIZE`
 - Such a chunk is called a **page**
- Actual page table layout is a bit more complex, three-level tree
 - root of tree is 4096-byte page table containing 512 PTEs (8 bytes or 64 bits per PTE)
 - first 9 bits of virtual address index top level page table, next mid level, etc.
 - Allows memory-efficient way of managing PTEs
 - Potentially requires 3 memory accesses for each VA → PA translation
 - RISC-V CPU uses **Translation Look-aside Buffer** (TLB)



RISC-V Paging Hardware (review 3)

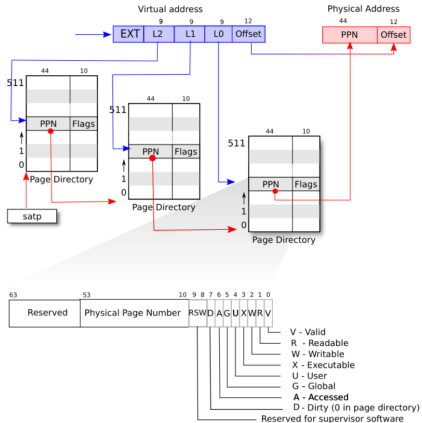


Figure 3: RISC-V address translation details (Cox et al., 2026, pg.33)

- PTE_V Valid, indicates whether the PTE is present, if not reference causes page fault
- PTE_R Readable, controls whether instructions are allowed to read the page
- PTE_W Writable, controls whether instructions are allowed to write to the page
- PTE_X Executable, controls whether CPU may interpret content of page as instructions and execute them
- PTE_U User, controls whether instructions in user mode are allowed to access page
- PTE_A Accessed, indicates if page has ever been accessed (read from) since created
- PTE_D Dirty, indicates if page has been written to



RISC-V Virtual Address Translation Question

- Assume following table shows page directory that maps to va 0x0000000000
- Assume commands are being run in user mode, unless otherwise specified
- Just given the physical page number, and flags (the 8 used flag bits)
- Then given a virtual address, determine if a fault occurs (and if so what type) or if not, what the actual physical address is.

Examples

- 1 VA 0x31FA (load from address to register)
- 2 VA 0x1010 (fetch instruction user mode)
- 3 VA 0x61A0 (load from address to register)
- 4 VA 0x00F0 (load from address to register)

Page	PPN (44)	Flags (10)
6	0x005F	0x72
5	0x0101	
4	0x000A	
3	0x223F	0x73
2	0x0007	
1	0x001F	0x7B
0	0x3FAF	0x63



Demand Paging and Page Eviction Algorithms

530 Comprehensive Exam Practice

Derek Harter

Department of Computer Science
East Texas A&M University

Fall 2026



EAST TEXAS A&M

Demand Paging to Disk and Page Eviction Algorithms

- Demand paging is often paired with paging to disk to provide unlimited virtual address space.
- Pages can be in secondary storage, PTE_V will be unset (invalid)
- Steady state in many environments is for memory to be full, so if a page fault occurs, need to select a page in memory to be evicted
 - Possibly need to write out first if dirty
- Page Eviction algorithms can have huge impact on performance, since eviction is costly
 - Want to select pages to evict that minimize page faults



Page Eviction Algorithms

- FIFO: treat memory as a first-in-first out queue, evict page that was in memory longest
 - Worst performance, will cause most page faults to occur of many eviction algorithms
 - Question: is xv6 memory allocation FIFO? Could it be easily turned into FIFO?
- LRU: least recently used, determine which physical frame hasn't been used for longest time
 - Best performance, but high overhead
 - e.g. How would we modify the xv6 memory allocator to support LRU?
- Optimal: pick page that won't be used for the longest amount of time in the future
 - This is only for simulations, we can't actually predict the future
- Clock: Modified FIFO, but use the access / use bit to try and select pages not recently used
 - This is strategy most modern operating systems use
 - The RISC-V Dirty and Accessed bits can be used for a clock eviction algorithm



Page Eviction Algorithms (FIFO, LRU, Optimal)

- Assume we have a physical memory with only 3 physical pages available (called physical frames)
- Given a stream of page references, show result of physical memory after each page reference

	0	1	2	3	4	5
Frame 1:	2					
Frame 2:	1					
Frame 3:	3					

Example page reference stream:

T1	2	3	4	5
4	2	1	4	2



Clock Page Eviction Algorithms

- ① Beginning at current frame pointer, scan until we find a frame with ($a=0$, $d=0$)
 - ② If a scan is made and no page meets (1), do a second scan for a page with ($a=0$, $d=1$). During this scan each access bit that is 1 is set to 0.
 - ③ If step (2) fails, the frame pointer has returned to the initial frame. Repeat step (1) and if necessary step (2).
- Frame pointer always points to frame after last one evicted.
 - When new page is loaded, $a=1$
 - Hardware sets $a=1$ on read/write
 - If a new page is loaded for write, $d=1$
 - Hardware sets $d=1$ on writes

Example page reference stream:

T1	2	3	4	5
4 (r)	2 (w)	1 (w)	4 (r)	2 (w)

	0	1	2	3	4	5
F1:	2					
	$a=1, d=0$					
F2:	1					
	$a=0, d=0$					
F3:	3					
	$a=0, d=1$					



References

Cox, R., Kaashoek, F., & Morris, R. (2026). *Xv6: A simple, unix-like teaching operating system*. Retrieved September 2, 2025, from <https://github.com/mit-pdos/xv6-riscv-book>

